

## Window Function Assignment(50 questions)

#1 find row number for each product ordered by price

with temp as

```
(select product_name,sum(price_at_sale) as total_amt from products join sale_items using(product_id)
group by product_name)
```

```
select product_name,total_amt,row_number()over(order by total_amt desc) as row_no from temp;
```

#2 find rank of products based on price

```
select product_name,price,rnk()over(order by price desc) as rank_no from products;
```

#3 find dense rank of products based on price

```
select product_name,price,dense_rnk()over(order by price desc) as rank_no from products;
```

#4 find top 3 highest priced products using rank

with temp as

```
(select product_name,price,rnk()over(order by price desc) as rank_no from products)
```

```
select product_name,price,rnk_no from temp where rank_no<4;
```

#5 find top 3 highest priced products using dense rank

with temp as

```
(select product_name,price,dense_rnk()over(order by price desc) as rank_no from products)
```

```
select product_name,price,rnk_no from temp where rank_no<4;
```

#6 find products with same price ranking

with temp as

```
(select product_name,price,dense_rnk()over(order by price desc) as rank_no from products),
temp2 as
```

```
(select rank_no,count(rank_no) as count_rank from temp group by rank_no)
```

```
select product_name,price,rnk_no from temp join temp2 using(rank_no) where count_rank>1;
```

#7 find row number of employees based on salary

```
select employee_name,salary,row_number()over(order by salary desc) as row_no from employees;
```

#8 find highest salary employee using rank

with temp as

```
(select employee_name,salary,dense_rnk()over(order by salary desc) as rank_no from employees)
```

```
select employee_name,salary from temp where rank_no=1;
```

#9 find second highest salary using dense rank

with temp as

```
(select employee_name,salary,dense_rnk()over(order by salary desc) as rank_no from employees)
```

```
select employee_name,salary from temp where rank_no=2;
```

#10 find third highest salary using rank

with temp as

```
(select employee_name,salary,dense_rnk()over(order by salary desc) as rank_no from employees)
```

```
select employee_name,salary from temp where rank_no=3;
```

#11 find total sales amount for each row using sum window  
select\*,sum(total\_amount)over() as total\_sales from sales;

#12 find running total of sales ordered by sale\_date  
select\*,sum(total\_amount) over(order by sale\_date) as running\_total from sales;

#13 find cumulative sum of sales for each customer  
select customer\_name,sale\_date,total\_amount,sum(total\_amount) over(partition by customer\_name  
order by sale\_date) as running\_total  
from customers join sales using(customer\_id);

#14 find average sales per employee using window function  
with temp as  
(select employee\_name,round(avg(total\_amount)) as avg\_amt  
from employees join sales using(employee\_id) group by employee\_id,employee\_name)  
select employee\_name,avg\_amt,round(sum(avg\_amt) over()) as total\_avg from temp;

#15 find max sale amount for each employee using window  
select employee\_name,total\_amount as total\_amt,max(total\_amount) over(partition by  
employee\_name) as max\_amt  
from employees join sales using(employee\_id);

#16 find min sale amount for each customer using window  
select customer\_name,total\_amount as total\_amt,min(total\_amount) over(partition by  
customer\_name) as max\_amt  
from customers join sales using(customer\_id);

#17 find difference between current sale and previous sale using lag  
select\*,total\_amount-lag(total\_amount) over() as diff from sales;

#18 find difference between current sale and next sale using lead  
select\*,total\_amount-lead(total\_amount) over() as diff from sales;

#19 find previous sale amount for each customer using lag  
select customer\_name,total\_amount,lag(total\_amount)over(partition by customer\_name) as prev\_amt  
from customers join sales using(customer\_id);

#20 find next sale amount for each customer using lead  
select customer\_name,total\_amount,lead(total\_amount)over(partition by customer\_name) as  
prev\_amt  
from customers join sales using(customer\_id);

#21 find first sale amount per customer using first\_value  
select customer\_name,sale\_date,total\_amount,first\_value(total\_amount) over(partition by  
customer\_name order by sale\_date) as first\_value\_  
from customers join sales using(customer\_id);

#22 find last sale amount per customer using last\_value

```
select customer_name,sale_date,total_amount,last_value(total_amount) over(partition by
customer_name order by sale_date) as last_value_
from customers join sales using(customer_id);
```

#23 find highest sale per customer using window function

```
select customer_name,total_amount,max(total_amount)over(partition by customer_name) as max_amt
from customers
join sales using(customer_id);
```

#24 find lowest sale per employee using window function

```
select employee_name,total_amount,min(total_amount)over(partition by employee_name) as min_amt
from employees
join sales using(employee_id);
```

#25 find percent rank of products based on price

```
select product_name,price,round(percent_rank() over(order by price),2) as percent_rank_value from
products;
```

#26 find second highest priced product using window function

```
with temp as
(select product_name,price,dense_rank()over(order by price desc) as rank_no from products)
select product_name,price from temp where rank_no=2;
```

#27 find products whose price is higher than previous product price

```
with temp as
(select product_name,price,lag(price)over() as prev_price from products)
select product_name,price,prev_price from temp where price>prev_price;
```

#28 find products whose price is lower than next product price

```
with temp as
(select product_name,price,lead(price)over() as next_price from products)
select product_name,price,next_price from temp where price<next_price;
```

#29 find difference between current product price and previous product price

```
with temp as
(select product_name,price,lag(price)over() as prev_price from products)
select product_name,price,prev_price,price-prev_price as diff from temp;
```

#30 find running total of product prices ordered by price

```
select product_name,price,sum(price) over(order by price) as running_total from products;
```

#31 find average product price per category using window

with temp as

```
(select category_name,product_name,sum(price_at_sale) as total_sales from categories join products
using(category_id)
```

```
join sale_items using(product_id) group by category_name,product_name),
```

temp\_2 as

```
(select category_name,product_name,total_sales,round(avg(total_sales)over(partition by
category_name),2)as avg_sales from temp)
```

```
select category_name,product_name,total_sales,avg_sales from temp_2;
```

#32 find products priced above category average

with temp as

```
(select category_name,product_name,price,round(avg(price)over(partition by category_name),2) as
avg_price_by_category from categories
```

```
join products using(category_id))
```

```
select category_name,product_name,price,avg_price_by_category from temp where
price>avg_price_by_category;
```

#33 find products priced below category average

with temp as

```
(select category_name,product_name,price,round(avg(price)over(partition by category_name),2) as
avg_price_by_category from categories
```

```
join products using(category_id))
```

```
select category_name,product_name,price,avg_price_by_category from temp where
price<avg_price_by_category;
```

#34 find highest priced product in each category using window

```
select category_name,product_name,price,max(price)over(partition by category_name) as
max_price_by_category from categories
```

```
join products using(category_id);
```

#35 find lowest priced product in each category using window

```
select category_name,product_name,price,min(price)over(partition by category_name) as
min_price_by_category from categories
```

```
join products using(category_id);
```

#36 find second highest priced product per category

with temp as

```
(select category_name,product_name,price,dense_rank()over(partition by category_name order by
price) as rank_no
```

```
from categories join products using(category_id))
```

```
select category_name,product_name,price from temp where rank_no=2;
```

#37 find total number of products per category using window

```
select category_name,product_name,price,count(product_name)over(partition by category_name) as
no_of_products_by_category from categories
```

```
join products using(category_id);
```

#38 find percentage contribution of each product price to total price

```
select product_name,price,round(price/sum(price) over() *100,2) as percent_contribution from products;
```

#39 find cumulative average price ordered by price

```
select product_name,price,round(avg(price) over(order by price), 2) as cumulative_avg from products;
```

#40 find difference between product price and average price

with temp as

```
(select product_name,price,round(avg(price)over(),2) as avg_price from products)
```

```
select product_name,price,avg_price,price-avg_price as diff from temp;
```

#41 find employees whose salary is higher than previous employee salary

with temp as

```
(select employee_name,salary,lag(salary)over() as prev_salary from employees)
```

```
select employee_name,salary,prev_salary from temp where salary>prev_salary;
```

#42 find difference between employee salary and next employee salary

with temp as

```
(select employee_name,salary,lead(salary)over() as next_salary from employees)
```

```
select employee_name,salary,next_salary,salary-next_salary as diff from temp;
```

#43 find running total of employee salaries

with temp as

```
(select employee_name,salary,sum(salary) over(order by employee_id) as running_total from employees)
```

```
select employee_name,salary,running_total from temp;
```

#44 find average salary per role using window

```
select employee_name,role,salary,round(avg(salary)over(partition by role),2) as avg_salary_by_role from employees;
```

#45 find employees earning above role average salary

with temp as

```
(select employee_name,role,salary,round(avg(salary)over(partition by role),2) as avg_salary_by_role from employees)
```

```
select employee_name,role,salary from temp where salary>avg_salary_by_role;
```

#46 find employees earning below role average salary

with temp as

```
(select employee_name,role,salary,round(avg(salary)over(partition by role),2) as avg_salary_by_role from employees)
```

```
select employee_name,role,salary from temp where salary<avg_salary_by_role;
```

#47 find highest salary per role using window

with temp as

(select employee\_name,role,salary,round(max(salary)over(partition by role),2) as max\_salary\_by\_role  
from employees)

select employee\_name,role,salary,max\_salary\_by\_role from temp;

#48 find second highest salary per role

with temp as

(select employee\_name,role,salary,dense\_rank() over(partition by role order by salary desc) as  
rank\_salary\_by\_role from employees)

select employee\_name,role,salary,rank\_salary\_by\_role from temp where rank\_salary\_by\_role=2;

#49 find customers whose total spending is greater than previous customer

with temp as

(select customer\_name,sum(total\_amount) as total\_amt from customers join sales using(customer\_id)  
group by customer\_name),

temp\_2 as

(select customer\_name,total\_amt,lag(total\_amt)over() as prev\_amt from temp)

select customer\_name,total\_amt,prev\_amt from temp\_2 where total\_amt>prev\_amt;

#50 find customers ranked by total spending using dense\_rank

select customer\_name,sum(total\_amount) as total\_amt,dense\_rank()over(order by  
sum(total\_amount)desc) as rank\_no

from customers join sales using(customer\_id) group by customer\_name;